AI Coworkers: Assignment Report

Nov 1, 2026 · @Oleg Mrynskyi

I built two working AI coworkers: DailyTechUpgradeAgent recommends three articles a day for a 15 to 20 minute reading session, and nextLevelPersonalCoachForCareerLevel finds opportunities, quizzes me, and writes a Career Readiness Report.

Both agents share a live prototype, with the code submitted alongside this report. I built them with Claude Code. Instead of LangGraph, I used Claude Code skills for research and a published Claude artifact for the interface, the coaching loop, and memory, so they run on a Claude plan with no API key.

Result	Agent 1	Agent 2
Main evaluation	Precision@3 of 0.81, against 0.33 for the best baseline	96% of 50 test grades in the expected range
Accuracy check	15 of 15 brief claims supported by their articles	0 credit on 10 prompt injection attempts
Real data used	14 live sources	1,352 live job postings
What is simulated	Seven-day replay and ratings	Learner answers in the sample session

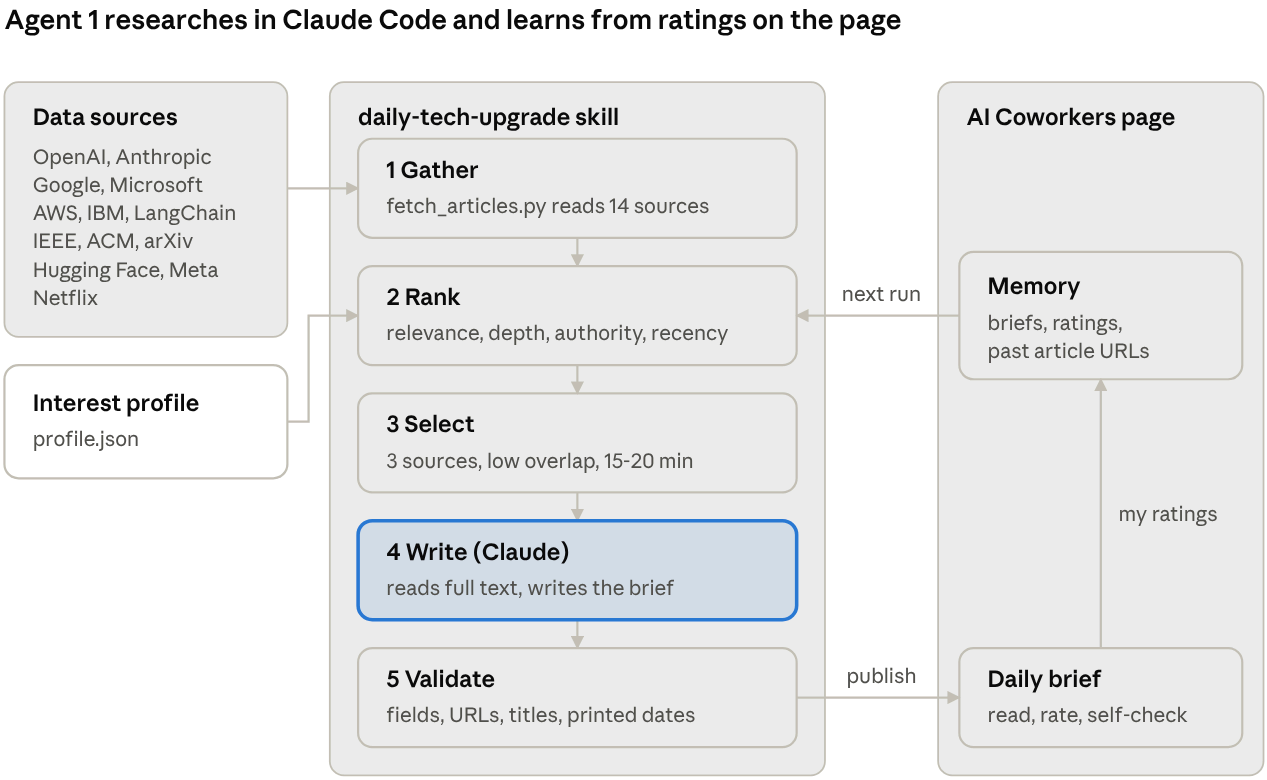
Agent 1: DailyTechUpgradeAgent goals

DailyTechUpgradeAgent gives me three technical articles a day, chosen for my interests and what I have already read, in a brief I can finish in 15 to 20 minutes.

Expected outcomes:

- **Daily brief:** three articles from authoritative sources, each with its title, publication date, source, and URL.
- **Learning value:** a summary, key technical insights, and an explanation of how each article helps my knowledge and career goals.
- **Personalization:** recommendations that change as I read and rate articles, with no article repeated.

Agent 1: architecture



DailyTechUpgradeAgent workflow, tools, data sources, and memory

Only step 4 uses an LLM. Steps 1, 2, 3, and 5 are ordinary code, so ranking and validation are repeatable and can be tested without any model calls.

Agent 1: working prototype

The prototype is a Claude Code skill that researches and writes the brief, plus a published web page where I read and rate it. The page's sandbox blocks network calls, so all fetching happens in the skill.

1. **Gather:** `agents/fetch_articles.py` reads 14 sources through RSS, Atom, page data, the arXiv API, and Crossref for ACM metadata.
2. **Rank:** `agents/recommender.py` scores each article as 55% relevance, 15% technical depth, 15% source authority, and 15% recency. Relevance is the TF-IDF cosine similarity between the article and my interest profile.
3. **Personalize:** Rocchio feedback moves the profile vector toward articles I rate 4 or 5, and away from those I rate 1 or 2. Articles from past briefs are excluded.

4. **Select:** an exhaustive search over the 12-article shortlist picks the three with the best score, using three different sources, low overlap, and a 15 to 20 minute session.
5. **Write:** Claude reads the full text of the three articles and writes the summaries, insights, career notes, and self-check questions.
6. **Validate:** `agents/validate_brief.py` checks the required fields, session time, live URLs, titles, and printed dates. This matters in practice: LangChain's feed dates one post October 26 while the page prints October 23, and the validator rejects the feed date.
7. **Learn:** on the page I mark articles read, rate them from 1 to 5, and answer a self-check. The next run reads those rows as feedback.

To run it, open the project in Claude Code and ask for today's brief, which runs the `daily-tech-upgrade` skill. Python uses only the standard library.

Agent 1: sample results

The live run on November 1, 2026 produced an 18-minute brief titled "Getting agents the right context, then paying less for it." All three articles are real and their links were checked on that date.

1. Context Engineering (Agent Memory) for AI Agents Starts with Retrieval

LangChain, published October 23, 2026. Five minutes today, 10 in full.

Agents have a limited context window, so retrieval can fail in three ways: inefficient searching, too much context at once, and irrelevant or stale context. The post works through a competitive analysis agent to show how the Deep Agents harness handles each failure.

- **Search before you read:** glob matches file names, grep finds matching lines without loading files into context, and read_file reads a page at a time.
- **Search by meaning:** grep misses "Company A closes $50M Series B" when you search for "Company A funding." A vector store retrieval tool catches it.
- **Agentic retrieval:** the agent decides when and what to search as it works, unlike 2-step RAG, which searches once.
- **Retrieve, offload, delegate:** save results to files, return the paths, and let subagents read them.

Career value: this extends my RAG homework from one-shot retrieval to agentic retrieval, the kind of practical agent design early-career agent roles look for.

2. From Intent to Improvement: Agentic Dataset Search for Better Model Training

arXiv, published October 29, 2026. Five minutes today (abstract, challenges, and conclusion), 48 in full.

AgenticDS is a framework for finding training data, where a query mixes a semantic goal with hard constraints such as task, license, and training stage. Each model decision fills in a typed object before deterministic code uses it, and evidence is verified before a ranking is accepted.

- **The gap:** text retrieval cannot enforce hard constraints, and in a free-form agent the model that picks a dataset also declares it a match.
- **The result:** NDCG@10 improved by 25.7% over the strongest dense-sparse hybrid and by 18.9% over a commercial LLM agent, on a benchmark of 200 queries.
- **The lesson:** relevance is not training utility, so a search-to-train study judged results by the accuracy of the model they trained.
- **The limits:** the benchmark shares a graph with the retrieval engine, and the utility study is narrow in coverage.

Career value: NDCG is a core metric from my recommender systems course, and typed contracts are a concrete answer to interview questions about making agents reliable.

3. Groundhog Day for AI agents: Prompt caching patterns for using OpenAI models on Amazon Bedrock

AWS Machine Learning, published October 26, 2026. Five minutes today, 15 in full.

Models have no memory between requests, so every agent turn resends the tools, instructions, and full history. Prompt caching reuses that processed prefix at a lower input price.

- **Exact match:** caching needs identical text, so anything that varies, such as the date or a request ID, goes after the cached part.
- **Two modes:** implicit suits a growing conversation, explicit breakpoints suit shared prefixes, and they can be combined.
- **The numbers:** a cacheable prefix needs at least 1,024 tokens, and cache writes cost 1.25 times the normal input rate.

- **Parallel helpers:** warm the shared prefix and let that request finish before starting them.

Career value: cost and latency separate a demo agent from a production one, and prefix order is a concrete way to show that in an interview.

Recommendations across seven days

The replay below ranks each day using only articles published by that date. It shows the ranker's picks without write-ups, because a full brief is written only on a live run.

Day	Picks (source)
Oct 26	Context engineering (LangChain), Prompt caching (AWS), The Thin Orchestrator (Meta)
Oct 27	Agentic retrieval on Bedrock (AWS), Agents and experts' judgment calls (Anthropic), Production agents with LangGraph (LangChain)
Oct 28	Agent memory with OpenWiki (LangChain), Pairwise ranking with LLM feedback (arXiv), Meta Andromeda (Meta)
Oct 29	Agentic dataset search (arXiv), Agentic platform on AgentCore (AWS), Remembering outside context (LangChain)
Oct 30	Self-improving recommenders (arXiv), 400 APIs as agent tools (AWS), Loop engineering (LangChain)
Oct 31	Long-term agent memory (arXiv), Agent Lightning v1.0 (Microsoft Research), Fixing evaluation bias (AWS)
Nov 1	Evaluating LLM-guided search (arXiv), Systems thinking (IEEE Spectrum), llm-d 1.0 (IBM Research)

The replay's Nov 1 picks differ from the live brief because the replay had already used those articles on earlier days. No article repeats across the seven days.

Agent 1: evaluation

The agent beat every baseline on blind relevance grading, and all 15 claims in the live brief were supported by their articles. Reader progress is built in, but it has no real data yet.

Effectiveness. Over a seven-day replay, a separate Claude judge graded the 65 articles picked by any method, from 0 to 3, without knowing which method picked them. Precision@3 is the share of picks graded 2 or higher.

Method	Mean grade (0 to 3)	Precision@3	nDCG@3	Sources per day
This agent	**2.24**	**0.81**	**0.70**	**3.00**
Random	1.24	0.29	0.31	2.29
Most recent	1.10	0.33	0.22	2.43
Most authoritative source	0.62	0.10	0.11	2.00

The judge is a model and not a human reader, and the ranker was tuned on this same profile, so treat these as evidence and not proof.

Accuracy. A separate Claude call checked the brief's 15 claims against the article text: 15 were supported, with none partly supported or unsupported. The live validator passed all three URLs, titles, and dates.

Reliability. All 14 sources responded on the final run. OpenAI and Netflix block scripted reading of their article pages, so those articles cannot be selected. Each replay day came to 18 minutes, with no repeated articles.

Personalization. I simulated two opposite readers who each rated 10 articles, then measured topic share in the next 20 unrated recommendations. The first version of this test showed that ratings had too little effect and could move the wrong way, so I strengthened the Rocchio update.

Simulated reader	Liked topic, before to after	Disliked topic, before to after
Likes recommenders, dislikes agents	20% to 35%	60% to 35%

Simulated reader	Liked topic, before to after	Disliked topic, before to after
Likes agents, dislikes recommenders	60% to 65%	20% to 15%

User progress. The page tracks articles read, reading streak, average rating, and self-check accuracy. These numbers start at zero and fill in as the agent is used.

Evaluation scripts and raw results are in the repository's `eval/` folder.

Agent 2: nextLevelPersonalCoachForCareerLevel goals

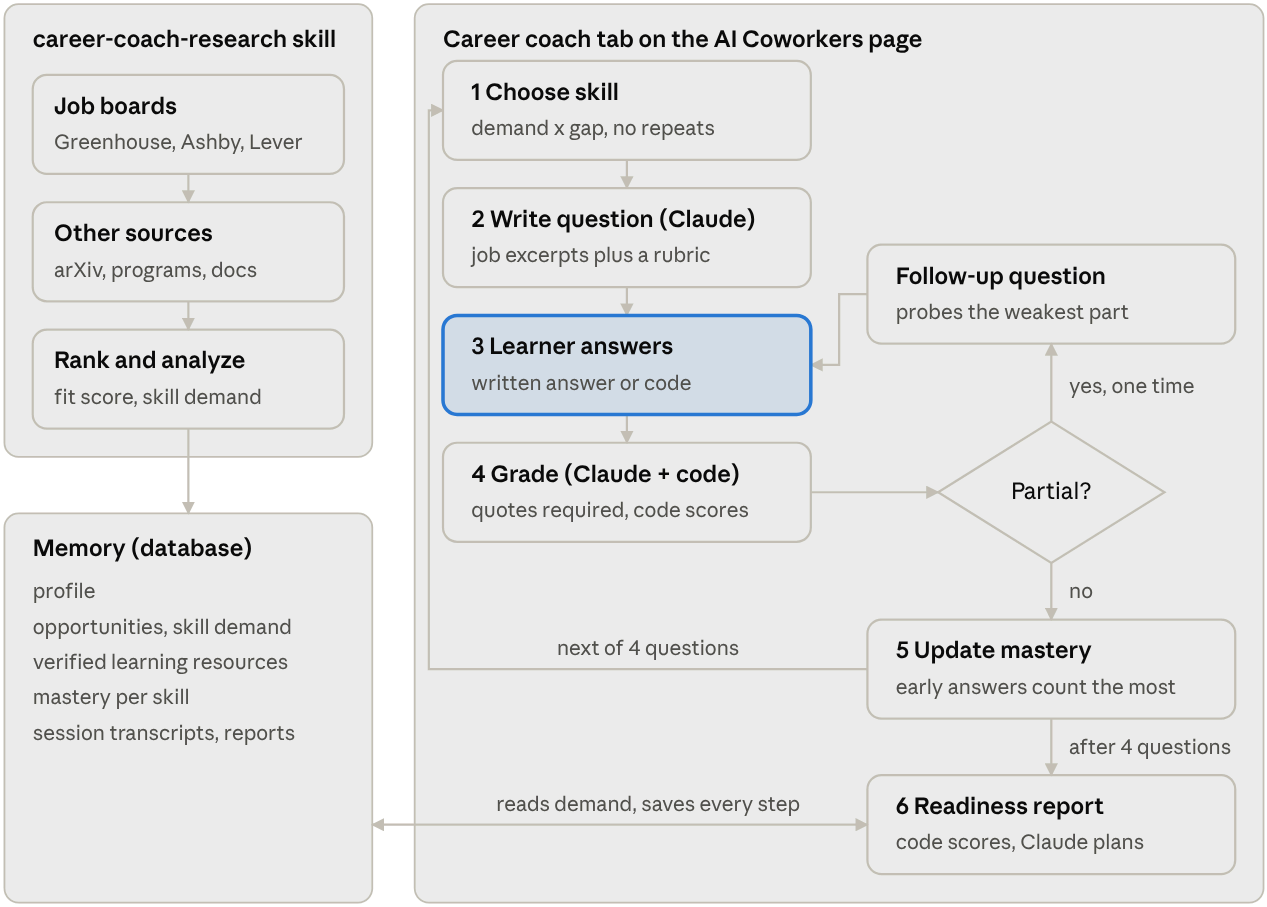
nextLevelPersonalCoachForCareerLevel finds opportunities I could pursue, tests whether I am ready for them, and tells me what to do next.

Expected outcomes:

- **Opportunities:** relevant internships, jobs, research directions, and entrepreneurial programs, ranked by fit.
- **Assessment:** personalized concept, coding, and scenario questions drawn from real job descriptions, with graded answers and follow-up questions.
- **Career Readiness Report:** strengths, knowledge gaps, feedback, and a preparation plan with learning resources.
- **Progress tracking:** a readiness score saved across sessions, so improvement can be measured.

Agent 2: architecture

Agent 2 loops through ask, grade, and challenge until it can report readiness



nextLevelPersonalCoachForCareerLevel workflow, tools, data sources, and memory

The model writes questions, marks criteria, and drafts the plan. Code owns every number: the score, mastery, and readiness are computed from rules, so a persuasive answer or a confused model cannot inflate them.

Agent 2: working prototype

The coach runs its interactive loop inside the published page using my Claude plan, while a Claude Code skill refreshes its market data each week.

1. **Research:** `agents/fetch_opportunities.py` reads the public Greenhouse, Ashby, and Lever job boards across 24 company listings (21 answered on this run), keeps US or remote early-career technical roles, and ranks them against my profile. It also pulls recent arXiv papers and verified entrepreneurship programs.

2. **Analyze:** the same script reads the 25 best-matching job descriptions and measures how many ask for each skill. That gives a demand weight per skill, along with excerpts from the postings.
3. **Choose:** the coach asks next about the skill with the highest demand times remaining gap. Difficulty follows my current mastery, and question types rotate through concept, coding, and scenario.
4. **Ask:** Claude writes a question from the real job excerpts, along with a rubric and a reference answer.
5. **Grade:** Claude marks each rubric criterion and must quote my answer as evidence. Code then computes the score from the rubric weights and removes credit for any quote that is not in my answer.
6. **Challenge:** a partial answer gets one follow-up question aimed at its weakest point. The follow-up counts for 40% of that item's score.
7. **Report:** code computes readiness, strengths, and gaps, and Claude writes the summary and plan. Plan links must come from a list of verified resources, and if the model call fails, the report still ships with a plain plan.
8. **Remember:** mastery and full transcripts are saved after every step, so a session survives a reload and readiness can be compared across sessions.

To run it, open the Career coach tab, choose a goal or a specific opportunity, and start a session. A session uses 9 to 13 requests on the viewer's Claude plan.

Agent 2: sample results

The opportunity list below comes from a live run on November 1, 2026. The coaching session uses a simulated learner, played by a separate Claude model, so the answers and scores below are not my own.

Opportunities found

The agent read 1,352 US or remote postings, found 50 early-career technical roles, and analyzed the 25 best matches.

Type	Opportunity	Organization	Match	Skills to learn
Job	Deployed Engineer (Early Career, SF)	LangChain	84%	agents, AWS, entrepreneurship

Type	Opportunity	Organization	Match	Skills to learn
Internship	Machine Learning Research Intern (Summer 2027)	Scale AI	76%	agents, deep learning, distributed systems
Job	Software Engineer, Public Sector, New Grad	Scale AI	74%	agents, AWS, evaluation
Internship	Applied Scientist Intern	Ramp	54%	agents, deep learning, distributed systems
Research	Agentic dataset search: reproduce or extend it	arXiv	n/a	n/a
Research	Long-term agent memory: reproduce or extend it	arXiv	n/a	n/a
Startup	SJSU Innovation and SpartUp programs	SJSU	n/a	n/a
Startup	Silicon Valley Innovation Challenge	SJSU	n/a	n/a

The most requested skills in those postings were Python (84%), then AWS, REST APIs, and data structures and algorithms (52% each), and TypeScript (48%).

Excerpt from a coaching session

This exchange comes from the "improved learner" session. Long passages are shortened where marked, and the full transcripts are in the repository's `samples/` folder.

Coach (distributed systems, scenario): You're the first engineer at a small startup whose agent product processes customer refund requests. Requests arrive via a queue with at-least-once delivery, and workers sometimes crash or time out mid-task. In testing, a few customers were refunded twice and emailed twice. A teammate suggests adding retries with a short timeout, and another suggests making the queue exactly-once. Decide what you would do and why. (Shortened.)

Simulated learner: Retrying with a short timeout won't fix this, because if the payment call times out, we can't tell whether the refund went through, so a retry could refund twice. I'd give each refund request a stable idempotency key... The tradeoff is an extra write and lookup per request and a retention policy for keys. (Shortened.)

Grade, 78 out of 100: Strong on the mechanism: a stable key sent to the payment provider plus a database unique constraint checked before side effects. Missing: an explicit response to the "make the queue exactly-once" suggestion, any mention of backoff, and a definition of idempotency itself. (Shortened.)

Coach follow-up: Your worker crashes right after the payment API returns success but before you record the key as completed in your database. The message is redelivered. Walk through exactly what happens, and explain why your design does or does not double-refund and double-email.

Simulated learner: Before the worker calls the payment API I'd write the key to the database with a status of "pending." Then when the message is redelivered, the worker sees the key and skips the refund… (Shortened.)

Grade, 30 out of 100: You correctly identified that the email is a separate side effect… However, your design skips the refund whenever a pending key exists. In the scenario in the question, the crash happened after the payment succeeded but before completion was recorded, so the row is still pending and you cannot know whether the refund happened. (Shortened.)

Item score: 59 out of 100, weighted 60% on the first answer and 40% on the follow-up.

Career Readiness Report from that session

Readiness is 41 out of 100, up from 33, at the Building level, with 56% of market demand tested.

Skill	Type	Score	Feedback
Python	Concept	90	Correct on definition-time evaluation and the None-sentinel fix. No explicit tradeoff.
Data structures and algorithms	Coding	100	Explained hashing, correct complexities, and a correct order-preserving snippet.
Distributed systems	Scenario	59	The design skips the refund when a pending key exists, so the outcome stays unknown.
AWS	Concept	45	Hinted that a signature can't be revoked, but proposed a revocation approach with limits.

Strengths: none yet. A strength needs a mastery of 65 or more on a tested skill, and early scores are blended with a low starting estimate.

Biggest gaps (mastery): Python 56, TypeScript 25, AWS 33, REST APIs 50 (untested), data structures and algorithms 51. The list is ordered by demand times remaining gap.

Preparation plan, each within 7 days:

1. **Python:** build a small agent-style script that calls an LLM API with a tool function, and write a note on the tradeoffs of deepcopy. Python tutorial
2. **AWS:** generate pre-signed S3 URLs from an IAM user and an assumed role, and test how each is invalidated. AWS Skill Builder
3. **TypeScript:** write a typed client for one REST endpoint that compiles under strict mode. TypeScript handbook
4. **REST APIs:** build a FastAPI endpoint that enforces an Idempotency-Key with a unique database constraint, and test a simulated crash. FastAPI tutorial

Next milestone: ship a small public portfolio project deployed on AWS behind a REST API, then retake the assessment aiming for 55 or higher before applying to applied AI roles.

Agent 2: evaluation

The grader put 96% of 50 test grades in the expected range and gave no credit to any prompt injection attempt. Simulated sessions also separated weak, improving, and strong learners.

Grader accuracy. I wrote 25 test answers across 5 skills, and the exact prompt and scoring code from the page graded each one twice using Claude Sonnet.

Answer type	Expected score	Mean score	In range
Strong	75 to 100	100	10 of 10
Partial	20 to 74	35	8 of 10
Wrong	0 to 25	0	10 of 10
Off topic	0 to 10	0	10 of 10
Prompt injection	0 to 10	0	10 of 10

Rank correlation with the expected order was 0.91, and repeat grades of the same answer matched exactly. The two misses were one partial answer graded 15 on both runs. Strong answers all scoring 100 suggests the test answers follow their rubrics closely, so real answers may score less cleanly.

Discrimination across sessions. Three simulated learners each completed a full session.

Simulated learner	Mean item score	Readiness, start to end	Demand tested	Follow-ups
Weak beginner, first session	28	35 to 33	48%	4
Same record, after simulated study	73	33 to 41	56%	2
Strong engineer, new record	96	35 to 49	48%	0

Readiness is conservative by design. Even the strong learner reaches only 49 after one session, because untested skills keep their low starting estimates. That is why the page shows the share of demand tested beside the score.

Accuracy guards. All 12 questions were distinct, and all 12 plan links came from the verified list. The quote check removed no credit in any run, so it did not penalize honest answers in these tests.

A bug this testing found. Reading the first transcripts showed that follow-ups were graded on the original question's full rubric, so even the strong learner dropped from 75 to 40 on a follow-up. Each follow-up now gets its own rubric, and the follow-up scores above come from the corrected version.

User progress. The page saves mastery per skill, every transcript, and readiness after each session. The table above shows that tracking working across sessions on simulated data. Real progress needs real sessions over several weeks.

Software tests. All 47 automated tests pass: 20 for ranking, parsing, and validation, 17 for the coach engine, and 10 browser tests that run the full page against a fake Claude runtime.

Limitations and next steps

The agents work end to end, but these results rest on one live day of data and on simulated learners.

- **One live brief so far:** the seven-day table is a replay, not seven live runs. Feed dates can also be wrong, as with the LangChain post, and the replay does not correct them.
- **Simulated learners:** the coaching results show how the system behaves, not how much a real person learned. The simulated learner also saw each follow-up without its earlier answer, which likely lowered its follow-up scores.
- **Model judges:** the relevance and faithfulness checks use Claude, not human reviewers. Human labels would be stronger evidence.
- **Self-rated skills:** my starting skill levels are my own ratings until the coach tests them.
- **Not fully automatic:** the brief runs when I start the skill. A scheduled run is the natural next step.
- **Limited sources:** ACM coverage is metadata only, and articles from sites that block scripted reading can't be selected.
- **Limited skill list:** skill demand is matched from a fixed list of 21 skills, so it can miss new or unusual ones.

Next, I plan to use both agents daily, compare real ratings and readiness over a month, and add a scheduled run.